# Circuitly API integration

# Every step. Every source line.

A code walkthrough of the dummy API integration in the Git Good Android project. Prepared 1 November 2026 from the current local files.

| Project | /Users/apple/Desktop/gitgood |
|---|---|
| API host | https://git-good-1.onrender.com |
| Operation | POST /api/sessions - multipart image + experiment_id + description |
| Actual capability | Upload connectivity test. ML1 and ML2 are stubs; no circuit correctness assessment is performed. |
| Document scope | 10 integration files; all source lines are numbered and explained. Full modified files include earlier auth/camera/layout code, explicitly identified as context. |
| Reading convention | The code column shows original source content with leading indentation normalized and long lines visually wrapped. Each numbered row still corresponds to exactly one physical source line. Blank lines, imports and delimiters are included. |

## What the user sees now

Choose an experiment, choose or capture an image, enter a description, then tap Continue. The app uploads the data and opens a conversation-style response screen. It shows loading, the submitted image/text, an actual server message and image receipt. Retry resubmits a failed upload. Exit conversation and system Back return Home, clear the image/description, and retain the selected experiment.

## Important boundary

No follow-up message API, experiment-list API, event/version contract, restoration endpoint or server-side close operation was present in the inspected deployment. The app does not invent those endpoints or generate fake circuit verdicts. The request ID in the receipt is not a restorable session ID.

# 1. Integration steps in implementation order

| Step | What was done |
| --- | --- |
| 1 - Inspect the service | Read /openapi.json and GET /health. Discover the actual /api/sessions path, required form fields, supported image formats, size limit and StubResponse shape. |
| 2 - Verify a real request | Send a synthetic 16 x 16 PNG to the live endpoint. Confirm connection_ok and the echoed experiment/image receipt before writing the Android request. |
| 3 - Define the client models | Add UploadReceipt and UploadRules. Represent the fields Android needs and reject invalid IDs/descriptions. |
| 4 - Build the upload boundary | Add UploadRepository. Read a bounded URI stream, inspect image metadata, create the multipart body, post over HTTPS, handle HTTP statuses and parse the receipt. |
| 5 - Add conversation state | Add ConversationUiState and ConversationViewModel. Track one draft, loading, result and error; preserve the draft for manual Retry. |
| 6 - Render the response | Add ConversationScreen with the submitted image/description, loading indicator, server receipt, errors, Retry and Exit. |
| 7 - Add a route | Add ScreenRoute.Conversation and declare its Navigation 3 entry in MainActivity. |
| 8 - Wire Continue | Add the callback through HomeDestination. MainActivity captures the selected experiment and Home draft, starts the upload and pushes Conversation. |
| 9 - Enforce form requirements | Update HomeScreen so description is required and limited to 2000 String units before Continue is enabled. Image format/size is checked in the repository. |
| 10 - Exit and clear | Cancel client work, clear conversation and Home draft, pop back to Home. Do not clear the selected experiment. |
| 11 - Verify and document | Run assembleDebug and testDebugUnitTest, add UploadRules tests, and write the runbook. Device UI testing remains a separate check. |

# 2. The real request and response contract

The inspected dummy service uses one multipart upload endpoint, not the larger proposed sessions/events design discussed earlier.

| Request part | Meaning and constraint |
|---|---|
| experiment_id | Required ID: 1-100 ASCII letters, digits, underscores or hyphens. The app currently sends local IDs such as ohms-law. |
| description | Required text. The app trims the draft before sending and rejects blank or more than 2000 Kotlin String units. |
| image | Required JPEG or PNG file, at most 5 MiB (5,242,880 bytes). |
| Transport | HTTPS POST to /api/sessions with multipart/form-data and an explicit boundary. No Firebase token or version field is attached. |

| Response field | How Android uses it |
|---|---|
| mode / status | Requires stub and connection_ok to accept the response. |
| request_id | Shown for debugging one request; not used to restore a conversation. |
| experiment_id / description | Stored as echoed form values. The screen uses the draft description and displays the echoed experiment ID. |
| message | Displayed exactly as the server explanation. |
| image | Contains content_type, size_bytes, width, height and sha256. The UI displays dimensions, bytes and hash. |
| ml1 / ml2 | Not parsed into diagnosis UI because they are stubs. |
| 413 / 415 / 422 | Mapped to image size, invalid image and invalid form messages respectively. |

## Reading multipart code

A boundary is a unique separator between fields. Content-Disposition identifies each part by its form name. The text \r\n in Kotlin strings becomes HTTP CRLF separators. The body is built as: text fields and image headers + raw image bytes + closing boundary. A file extension is chosen from the decoded media type, not a user-provided filename.

# 3. How state and navigation connect

| Layer | Responsibility and handoff |
| --- | --- |
| HomeScreen | Displays the form and emits onContinueClick. It does not call the API. |
| HomeDestination | Passes onContinue from MainActivity into HomeScreen; retains prior picker/camera setup. |
| MainActivity | Reads selected experiment and Home state. Calls ConversationViewModel.start, then pushes the Conversation route. |
| ConversationViewModel | Sets loading, launches work in viewModelScope, calls UploadRepository and exposes StateFlow updates. |
| UploadRepository | Runs blocking URI/network work on Dispatchers.IO and returns UploadReceipt or throws. |
| ConversationScreen | Collects no network data itself. MainActivity supplies state; recomposition displays progress, result or errors. |
| Retry | Uses the captured draft for a fresh upload attempt. This service has no idempotency or retry-event API. |
| Exit / Back | Cancels the local Job, clears conversation and Home state, and pops the route. An upload already received by the server cannot be undone. |

## Unchanged prerequisites

Firebase login, Internet permission, Hilt, Navigation 3, Coil preview, the five-experiment list, HomeViewModel, the camera provider and image picker were already in place. No API-specific Gradle dependency, manifest permission or Firebase configuration change was needed. This document includes complete modified files, so earlier functionality appears in their annotated context.

# Source file index

| File / status | Path and responsibility |
| --- | --- |
| 01. UploadReceipt.kt (new, 25 lines) | app/src/main/java/com/example/gitgood/data/model/UploadReceipt.kt - Response model and upload constraints |
| 02. UploadRepository.kt (new, 87 lines) | app/src/main/java/com/example/gitgood/data/repository/UploadRepository.kt - HTTPS multipart upload |
| 03. ConversationViewModel.kt (new, 63 lines) | app/src/main/java/com/example/gitgood/viewModel/ConversationViewModel.kt - Submission and UI state |
| 04. ConversationScreen.kt (new, 66 lines) | app/src/main/java/com/example/gitgood/ui/theme/screen/conversation/ConversationScreen.kt - Conversation-style result UI |
| 05. MainActivity.kt (modified, 175 lines) | app/src/main/java/com/example/gitgood/MainActivity.kt - Navigation and orchestration |
| 06. ScreenRoute.kt (modified, 18 lines) | app/src/main/java/com/example/gitgood/navigation/ScreenRoute.kt - Typed route definition |
| 07. AppNavigation.kt (modified, 122 lines) | app/src/main/java/com/example/gitgood/navigation/AppNavigation.kt - Home callback bridge |
| 08. HomeScreen.kt (modified, 204 lines) | app/src/main/java/com/example/gitgood/ui/theme/screen/home/HomeScreen.kt - Required inputs and submission button |
| 09. UploadRulesTest.kt (new, 15 lines) | app/src/test/java/com/example/gitgood/data/model/UploadRulesTest.kt - Input-validation tests |
| 10. dummy-api-integration.md (new, 52 lines) | docs/dummy-api-integration.md - Integration runbook |

# 4. UploadReceipt.kt

app/src/main/java/com/example/gitgood/data/model/UploadReceipt.kt

New for API integration. Start here to learn the wire fields and local validation limits.

Snapshot SHA-256: 0e1f2df941840856f338a6118ab45df06a750260e535e7bf6a1faa0850203478

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.data.model` | Places this file in the com.example.gitgood.data.model Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `/** Connectivity receipt, not an ML diagnosis or a restorable backend session. */` | Documents that this object proves upload connectivity only; it is neither an ML verdict nor a backend session. |
| 4 | `data class UploadReceipt(` | Defines an immutable Kotlin data class. Constructor properties also receive generated copy, equality and toString methods. |
| 5 | `val requestId: String,` | Stores the unique server request identifier for this upload receipt. |
| 6 | `val experimentId: String,` | Stores the experiment identifier echoed by the server. |
| 7 | `val description: String,` | Stores the submitted description echoed in the response. |
| 8 | `val message: String,` | Stores the human-readable server message displayed in the response bubble. |
| 9 | `val contentType: String,` | Stores the detected image media type returned by the server. |
| 10 | `val sizeBytes: Long,` | Stores the received file size in bytes as a Long. |
| 11 | `val width: Int,` | Stores received image width in pixels. |
| 12 | `val height: Int,` | Stores received image height in pixels. |
| 13 | `val sha256: String` | Stores the server-reported image SHA-256. The current client displays it but does not compute and compare a local hash. |
| 14 | `)` | Closes the current argument or property list. |
| 15 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 16 | `object UploadRules {` | Creates a single shared holder for validation constants and a text-validation function. |
| 17 | `const val MAX_IMAGE_BYTES = 5 * 1024 * 1024` | Sets the image limit to 5 x 1024 x 1024 bytes: 5 MiB. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 18 | `const val MAX_DESCRIPTION_LENGTH = 2000` | Names the 2000-character description limit used by repository validation. |
| 19 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 20 | `fun validate(experimentId: String, description: String) {` | Validates the experiment identifier and description before a request can be made. |
| 21 | `require(experimentId.matches(Regex("[A-Za-z0-9_-]{1,100}"))) { "Select a valid experiment." }` | Allows only 1-100 ASCII letters, digits, underscores or hyphens in the experiment ID; failure throws IllegalArgumentException. |
| 22 | `require(description.isNotBlank()) { "Add a description before continuing." }` | Rejects empty or whitespace-only descriptions with a user-readable message. |
| 23 | `require(description.length <= MAX_DESCRIPTION_LENGTH) { "Keep the description within 2000 characters." }` | Rejects descriptions longer than the Kotlin String length limit. Kotlin length counts UTF-16 code units, which can be stricter than Unicode character counting. |
| 24 | `}` | Closes the current Kotlin block or type. |
| 25 | `}` | Closes the current Kotlin block or type. |

# 5. UploadRepository.kt

app/src/main/java/com/example/gitgood/data/repository/UploadRepository.kt

New for API integration. This is the only new component that performs the real HTTP request. It uses Android/platform APIs, so no networking dependency was added.

Snapshot SHA-256: be121c08f5adfe2d4ab9344c7722204f41fffe3c16379da47283c2d305419573

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.data.repository` | Places this file in the com.example.gitgood.data.repository Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import android.content.Context` | Imports Android access to services and content providers. |
| 4 | `import android.graphics.BitmapFactory` | Imports Android image-header decoding used to inspect format and dimensions. |
| 5 | `import android.net.Uri` | Imports Android identifier for a selected or captured image. |
| 6 | `import com.example.gitgood.data.model.UploadReceipt` | Imports the typed connectivity response model. |
| 7 | `import com.example.gitgood.data.model.UploadRules` | Imports shared text/file-size validation rules. |
| 8 | `import dagger.hilt.android.qualifiers.ApplicationContext` | Imports the Hilt qualifier selecting the application Context. |
| 9 | `import kotlinx.coroutines.Dispatchers` | Imports coroutine execution contexts, including the IO dispatcher. |
| 10 | `import kotlinx.coroutines.ensureActive` | Imports cooperative coroutine cancellation checks. |
| 11 | `import kotlinx.coroutines.withContext` | Imports switching coroutine work to another execution context. |
| 12 | `import org.json.JSONObject` | Imports Android JSON object parsing. |
| 13 | `import java.io.ByteArrayOutputStream` | Imports bounded in-memory collection of the source file bytes. |
| 14 | `import java.io.IOException` | Imports readable file/network failure exceptions. |
| 15 | `import java.net.HttpURLConnection` | Imports the platform HTTP request/response implementation. |
| 16 | `import java.net.URL` | Imports the API endpoint URL representation. |
| 17 | `import java.util.UUID` | Imports random multipart boundary generation. |
| 18 | `import javax.inject.Inject` | Imports constructor injection recognized by Hilt. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 19 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 20 | `class UploadRepository @Inject constructor(@ApplicationContext private val context: Context) {` | Hilt injects the application Context, avoiding storage of an Activity. The repository uses it to open content URIs. |
| 21 | `companion object {` | Starts class-level constants. |
| 22 | `const val BASE_URL = "https://git-good-1.onrender.com"` | Sets the HTTPS host of the dummy API; no Firebase token is sent to this endpoint. |
| 23 | `}` | Closes the current Kotlin block or type. |
| 24 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 25 | `suspend fun upload(experimentId: String, description: String, uri: Uri): UploadReceipt = withContext(Dispatchers.IO) {` | Declares a suspend upload operation returning UploadReceipt. withContext(IO) moves blocking file/network work off the main thread. |
| 26 | `UploadRules.validate(experimentId, description)` | Checks experiment-ID syntax and description limits again, independently of the UI. |
| 27 | `val bytes = context.contentResolver.openInputStream(uri)?.use { input ->` | Opens the selected content URI through ContentResolver. use closes the image stream when finished. |
| 28 | `val output = ByteArrayOutputStream()` | Collects the bounded file bytes in memory; this is not a bitmap or decoded pixel array. |
| 29 | `val buffer = ByteArray(8192)` | Allocates an 8192-byte buffer for reading the source stream. |
| 30 | `while (true) {` | Reads repeatedly until the image stream ends. |
| 31 | `ensureActive()` | Stops cooperative file-reading work if the upload coroutine has been cancelled. |
| 32 | `val count = input.read(buffer)` | Reads the next chunk and records the actual byte count. |
| 33 | `if (count < 0) break` | A negative byte count means end-of-stream; leave the loop. |
| 34 | `require(output.size() + count <= UploadRules.MAX_IMAGE_BYTES) { "Choose an image smaller than 5 MiB." }` | Rejects the read before adding a chunk that would exceed 5 MiB. Exactly 5 MiB is accepted despite the wording "smaller than". |
| 35 | `output.write(buffer, 0, count)` | Copies only the bytes read, not the unused portion of the buffer. |
| 36 | `}` | Closes the current Kotlin block or type. |
| 37 | `output.toByteArray()` | Produces the byte array that will form the multipart image part. |
| 38 | `} ?: throw IOException("Cannot read this image. Please select it again.")` | Throws a readable error when ContentResolver cannot supply an input stream. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 39 | `val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }` | Configures an image-bounds-only decode so validation does not allocate the full image pixel buffer. |
| 40 | `BitmapFactory.decodeByteArray(bytes, 0, bytes.size, bounds)` | Inspects the image bytes to populate dimensions and detected media type. |
| 41 | `val mime = bounds.outMimeType` | Reads the decoder-detected MIME type rather than trusting a filename extension. |
| 42 | `require(mime == "image/jpeg" \|\| mime == "image/png") { "Choose a JPEG or PNG image." }` | Accepts JPEG and PNG; rejects HEIC, GIF and other image types for this server contract. |
| 43 | `require(bounds.outWidth > 0 && bounds.outHeight > 0) { "This image cannot be read." }` | Rejects a file whose image dimensions cannot be decoded. |
| 44 | `val boundary = "Circuitly-${UUID.randomUUID()}"` | Generates a random multipart boundary separating form sections. This UUID is not a retry/idempotency event ID. |
| 45 | `val connection = URL("$BASE_URL/api/sessions").openConnection() as HttpURLConnection` | Opens the API URL as an HTTP connection. Because the URL uses HTTPS, transport uses TLS. |
| 46 | `try {` | Starts a block whose finally clause will release the connection on success or failure. |
| 47 | `connection.requestMethod = "POST"` | Uses the POST operation advertised by the OpenAPI specification. |
| 48 | `connection.connectTimeout = 90_000` | Allows up to 90 seconds to establish the connection, including potential server startup delay. |
| 49 | `connection.readTimeout = 90_000` | Allows up to 90 seconds for a blocking response read; this is not a guaranteed total request deadline. |
| 50 | `connection.instanceFollowRedirects = false` | Does not automatically follow redirects, so the multipart payload is not silently forwarded to another URL. |
| 51 | `connection.doOutput = true` | Allows writing a request body. |
| 52 | `connection.setRequestProperty("Accept", "application/json")` | Requests a JSON response through the Accept header. |
| 53 | `connection.setRequestProperty("Content-Type", "multipart/form-data; boundary=$boundary")` | Declares multipart/form-data and the exact boundary used below. |
| 54 | `val fields = buildString {` | Builds the text headers and fields that come before the raw image bytes. |
| 55 | `append("--$boundary\r\nContent-Disposition: form-data; name=\"experiment_id\"\r\n\r\n$experimentId\r\n")` | Adds the experiment_id form part. Literal \r\n sequences in source become CRLF separators in the HTTP body. |
| 56 | `append("--$boundary\r\nContent-Disposition: form-data; name=\"description\"\r\n\r\n$description\r\n")` | Adds the description form part as UTF-8 text. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 57 | `append("--$boundary\r\nContent-Disposition: form-data; name=\"image\"; filename=\"circuit.${if (mime == "image/png") "png" else "jpg"}\"\r\n")` | Adds the image part disposition with a generated filename and extension matching the detected MIME type. |
| 58 | `append("Content-Type: $mime\r\n\r\n")` | Adds the image media type and a blank CRLF line before the binary image data. |
| 59 | `}.toByteArray(Charsets.UTF_8)` | Encodes multipart text as UTF-8 bytes. |
| 60 | `val end = "\r\n--$boundary--\r\n".toByteArray(Charsets.UTF_8)` | Builds the closing multipart boundary after the image bytes. |
| 61 | `connection.setFixedLengthStreamingMode(fields.size + bytes.size + end.size)` | Declares the exact request-body length so HttpURLConnection can stream a fixed-size body. |
| 62 | `ensureActive()` | Checks for cancellation before starting the request write. |
| 63 | `connection.outputStream.use { it.write(fields); it.write(bytes); it.write(end) }` | Writes form headers/text, image bytes, then the final boundary. use closes the request output stream. |
| 64 | `val status = connection.responseCode` | Reads the HTTP status code, waiting for the response if necessary. |
| 65 | `if (status !in 200..299) {` | Treats only 2xx responses as success. |
| 66 | `throw IOException(when (status) {` | Converts known HTTP error statuses into readable IOException messages. |
| 67 | `413 -> "The server rejected the image: maximum size is 5 MiB."` | Maps 413 to the server image-size rejection. |
| 68 | `415 -> "The server could not read the image. Use JPEG or PNG."` | Maps 415 to unsupported/invalid image content. |
| 69 | `422 -> "The server rejected the form. Check the experiment, description, and image."` | Maps 422 to missing or invalid form fields. |
| 70 | `else -> "Server error ($status). Please try again."` | Reports the status code for any other error, including redirects and 5xx responses. |
| 71 | `})` | Closes the lambda and its surrounding call. |
| 72 | `}` | Closes the current Kotlin block or type. |
| 73 | `val json = connection.inputStream.bufferedReader(Charsets.UTF_8).use { JSONObject(it.readText()) }` | Reads the successful response as UTF-8, closes its stream, and parses it with Android JSONObject. |
| 74 | `require(json.getString("mode") == "stub" && json.getString("status") == "connection_ok") {` | Checks that this is the expected stub response with connection_ok status. Other modes require an intentional client update. |
| 75 | `"Unexpected API response. This client expects the connectivity-test API."` | Provides the failure message when the response is not the connectivity-test format. |
| 76 | `}` | Closes the current Kotlin block or type. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 77 | `val image = json.getJSONObject("image")` | Opens the nested image receipt JSON object. |
| 78 | `UploadReceipt(` | Constructs the strongly named response model returned to the ViewModel. |
| 79 | `json.getString("request_id"), json.getString("experiment_id"), json.getString("description"),` | Reads required request ID, experiment ID and description fields; getString throws when required data is missing. |
| 80 | `json.getString("message"), image.getString("content_type"), image.getLong("size_bytes"),` | Reads the server message, image media type and received byte count. |
| 81 | `image.getInt("width"), image.getInt("height"), image.getString("sha256")` | Reads width, height and server SHA-256 to complete the receipt. |
| 82 | `)` | Closes the current argument or property list. |
| 83 | `} finally {` | Runs cleanup regardless of the return value or exception. |
| 84 | `connection.disconnect()` | Releases the HttpURLConnection. Cancellation cannot guarantee that the server did not already receive the upload. |
| 85 | `}` | Closes the current Kotlin block or type. |
| 86 | `}` | Closes the current Kotlin block or type. |
| 87 | `}` | Closes the current Kotlin block or type. |

# 6. ConversationViewModel.kt

app/src/main/java/com/example/gitgood/viewModel/ConversationViewModel.kt

New for API integration. This coordinates one upload-test draft, progress, result, errors and manual retry.

Snapshot SHA-256: d8e1c005a771bde986caf41b3467610ef2bcc4e92700424b2b768da9e3fc932c

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.viewModel` | Places this file in the com.example.gitgood.viewModel Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import android.net.Uri` | Imports Android identifier for a selected or captured image. |
| 4 | `import androidx.lifecycle.ViewModel` | Imports Android lifecycle-aware state holder. |
| 5 | `import androidx.lifecycle.viewModelScope` | Imports the ViewModel-owned coroutine scope. |
| 6 | `import com.example.gitgood.data.model.UploadReceipt` | Imports the typed connectivity response model. |
| 7 | `import com.example.gitgood.data.model.UploadRules` | Imports shared text/file-size validation rules. |
| 8 | `import com.example.gitgood.data.repository.UploadRepository` | Imports UploadRepository so this file can use that existing app class or composable by its short name. |
| 9 | `import dagger.hilt.android.lifecycle.HiltViewModel` | Imports Hilt ViewModel generation. |
| 10 | `import kotlinx.coroutines.CancellationException` | Imports cancellation that must not be treated as an ordinary error. |
| 11 | `import kotlinx.coroutines.Job` | Imports a handle for cancelling pending upload work. |
| 12 | `import kotlinx.coroutines.flow.MutableStateFlow` | Imports the privately writable observable state. |
| 13 | `import kotlinx.coroutines.flow.asStateFlow` | Imports a read-only observable view of MutableStateFlow. |
| 14 | `import kotlinx.coroutines.launch` | Imports starting a coroutine. |
| 15 | `import java.net.SocketTimeoutException` | Imports specific handling of socket timeouts. |
| 16 | `import javax.inject.Inject` | Imports constructor injection recognized by Hilt. |
| 17 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 18 | `data class ConversationUiState(` | Defines all observable state for one upload-test conversation. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 19 | `val experimentId: String = "",` | Defaults the API experiment ID to empty before a submission exists. |
| 20 | `val experimentName: String = "",` | Keeps the display name separately from the API identifier. |
| 21 | `val imageUri: Uri? = null,` | Stores a nullable local image URI for the user message preview. |
| 22 | `val description: String = "",` | Stores the submitted description. |
| 23 | `val isLoading: Boolean = false,` | Tracks whether upload work is currently pending. |
| 24 | `val receipt: UploadReceipt? = null,` | Holds the parsed successful receipt; null means none has been received. |
| 25 | `val error: String? = null` | Holds a user-visible failure message. |
| 26 | `)` | Closes the current argument or property list. |
| 27 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 28 | `@HiltViewModel` | Registers the next ViewModel class for Hilt-generated construction. |
| 29 | `class ConversationViewModel @Inject constructor(private val repository: UploadRepository) : ViewModel() {` | Declares a Hilt ViewModel and injects the network repository through its constructor. |
| 30 | `private val _uiState = MutableStateFlow(ConversationUiState())` | Owns a mutable StateFlow internally, initialized to an empty conversation. |
| 31 | `val uiState = _uiState.asStateFlow()` | Exposes read-only StateFlow so UI code can observe but cannot directly replace state. |
| 32 | `private var uploadJob: Job? = null` | Keeps the coroutine Job so Exit can cancel pending client work. |
| 33 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 34 | `fun start(id: String, name: String, image: Uri, description: String) {` | Starts a new upload draft using the currently selected experiment, image and description. |
| 35 | `if (_uiState.value.isLoading) return` | Ignores another start request while an upload is already running. |
| 36 | `_uiState.value = ConversationUiState(id, name, image, description.trim())` | Copies the draft into state and trims leading/trailing whitespace from the description. |
| 37 | `retry()` | Starts the first upload using the same mechanism as Retry. |
| 38 | `}` | Closes the current Kotlin block or type. |
| 39 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 40 | `fun retry() {` | Provides an explicit manual retry operation; no automatic network retry is installed. |
| 41 | `val draft = _uiState.value` | Captures the current draft in a stable local variable for this attempt. |
| 42 | `if (draft.isLoading \|\| draft.imageUri == null) return` | Avoids duplicate in-flight uploads and refuses to upload without an image URI. |
| 43 | `_uiState.value = draft.copy(isLoading = true, error = null)` | Preserves the draft while setting loading true and clearing its previous error. |
| 44 | `uploadJob = viewModelScope.launch {` | Launches a coroutine tied to the ViewModel lifetime and retains its Job. |
| 45 | `try {` | Catches validation and network failures while preserving cancellation semantics. |
| 46 | `UploadRules.validate(draft.experimentId, draft.description)` | Checks the text fields before opening the image/network operation. |
| 47 | `val receipt = repository.upload(draft.experimentId, draft.description, draft.imageUri)` | Calls the suspend repository operation and waits for a parsed response. |
| 48 | `_uiState.value = draft.copy(receipt = receipt)` | Stores the receipt using the captured draft; loading is false because the captured draft was not loading. |
| 49 | `} catch (e: CancellationException) {` | Handles coroutine cancellation separately from ordinary errors. |
| 50 | `throw e` | Rethrows cancellation instead of showing a misleading upload failure. |
| 51 | `} catch (_: SocketTimeoutException) {` | Handles connection/read timeout errors with a tailored message. |
| 52 | `_uiState.value = draft.copy(error = "The server took too long to respond. Retry when it is available.")` | Preserves the user submission and exposes a timeout message; loading returns to false. |
| 53 | `} catch (e: Exception) {` | Handles all remaining ordinary exceptions, including invalid input, URI-read failures, HTTP errors and JSON parsing. |
| 54 | `_uiState.value = draft.copy(error = e.localizedMessage ?: "Upload failed. Check your connection and retry.")` | Shows the available exception message or a generic connection fallback while preserving the draft. |
| 55 | `}` | Closes the current Kotlin block or type. |
| 56 | `}` | Closes the current Kotlin block or type. |
| 57 | `}` | Closes the current Kotlin block or type. |
| 58 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 59 | `fun exit() {` | Ends the local test conversation; there is no backend close-session endpoint in this dummy service. |
| 60 | `uploadJob?.cancel()` | Requests cancellation of pending client work. It cannot undo an upload that already reached the server. |
| 61 | `_uiState.value = ConversationUiState()` | Clears the local draft, loading state, response and error. |
| 62 | `}` | Closes the current Kotlin block or type. |
| 63 | `}` | Closes the current Kotlin block or type. |

# 7. ConversationScreen.kt

app/src/main/java/com/example/gitgood/ui/theme/screen/conversation/ConversationScreen.kt

New for API integration. A user submission bubble and actual server receipt are displayed. This is not a working multi-turn chatbot.

Snapshot SHA-256: 1134448874f4105f1ea0957815e729d6f5db118a4a69f2e75383b1dadff243c2

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.ui.theme.screen.conversation` | Places this file in the com.example.gitgood.ui.theme.screen.conversation Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import androidx.compose.foundation.background` | Imports background color modifiers. |
| 4 | `import androidx.compose.foundation.layout.*` | Imports the members of androidx.compose.foundation.layout used by the UI or state code in this file. |
| 5 | `import androidx.compose.foundation.rememberScrollState` | Imports remembered scroll offsets. |
| 6 | `import androidx.compose.foundation.shape.RoundedCornerShape` | Imports rounded cards/buttons/input fields. |
| 7 | `import androidx.compose.foundation.verticalScroll` | Imports scrollable column behavior. |
| 8 | `import androidx.compose.material3.*` | Imports the members of androidx.compose.material3 used by the UI or state code in this file. |
| 9 | `import androidx.compose.runtime.Composable` | Imports Compose UI function annotation. |
| 10 | `import androidx.compose.ui.Alignment` | Imports alignment options for Compose layouts. |
| 11 | `import androidx.compose.ui.Modifier` | Imports layout, appearance and behavior modifiers. |
| 12 | `import androidx.compose.ui.layout.ContentScale` | Imports how an image is fitted to its display bounds. |
| 13 | `import androidx.compose.ui.text.font.FontWeight` | Imports text emphasis options. |
| 14 | `import androidx.compose.ui.unit.dp` | Imports density-independent layout dimensions. |
| 15 | `import androidx.compose.ui.unit.sp` | Imports font-size dimensions. |
| 16 | `import coil3.compose.AsyncImage` | Imports Coil image rendering from a content URI. |
| 17 | `import com.example.gitgood.ui.theme.*` | Imports the members of com.example.gitgood.ui.theme used by the UI or state code in this file. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 18 | `import com.example.gitgood.viewModel.ConversationUiState` | Imports ConversationUiState so this file can use that existing app class or composable by its short name. |
| 19 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 20 | `@Composable` | Marks the next function as Compose UI content. |
| 21 | `fun ConversationScreen(state: ConversationUiState, onRetry: () -> Unit, onExit: () -> Unit) {` | Declares a stateless Compose screen: it receives state and emits Retry/Exit callbacks; it never performs HTTP itself. |
| 22 | `Column(Modifier.fillMaxSize().background(BackgroundColor).verticalScroll(rememberScrollState())` | Creates a full-screen, theme-colored, scrollable vertical layout. |
| 23 | `.padding(horizontal = 24.dp, vertical = 12.dp), verticalArrangement = Arrangement.spacedBy(18.dp)) {` | Applies outer padding and spacing between conversation sections. |
| 24 | `Row(Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically) {` | Starts a horizontal header row with vertically centered content. |
| 25 | `Text("Circuitly", Modifier.weight(1f), color = PrimaryGreen, fontSize = 18.sp, fontWeight = FontWeight.SemiBold)` | Shows the Circuitly brand and lets it consume remaining row width. |
| 26 | `TextButton(onClick = onExit) { Text("Exit conversation", color = PrimaryGreen) }` | Connects the visible Exit conversation control to the supplied navigation/cleanup callback. |
| 27 | `}` | Closes the current Kotlin block or type. |
| 28 | `Text(state.experimentName.ifBlank { "Connection test" }, fontSize = 22.sp, color = PrimaryTextColor)` | Displays the experiment name, with Connection test as an empty-state fallback. |
| 29 | `Text("Dummy API · Upload test only. No circuit diagnosis is performed.", color = SecondaryTextColor, fontSize = 13.sp)` | Explicitly identifies the response as a dummy upload test without circuit diagnosis. |
| 30 | `if (state.imageUri != null) {` | Only renders a user-message image card when a draft URI exists. |
| 31 | `Surface(Modifier.fillMaxWidth(), shape = RoundedCornerShape(14.dp), color = LightGreen) {` | Creates the user-message bubble using the theme light-green color. |
| 32 | `Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {` | Adds inner padding and spacing to the user-message card. |
| 33 | `Text("You", fontWeight = FontWeight.SemiBold, color = PrimaryGreen)` | Labels the card as the user submission. |
| 34 | `AsyncImage(state.imageUri, "Submitted circuit", Modifier.fillMaxWidth().height(200.dp), contentScale = ContentScale.Fit)` | Uses Coil to load the local URI into a 200dp-high image, fitting the whole image inside its bounds. |
| 35 | `Text(state.description, color = PrimaryTextColor)` | Displays the description that was sent with the image. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 36 | `}` | Closes the current Kotlin block or type. |
| 37 | `}` | Closes the current Kotlin block or type. |
| 38 | `}` | Closes the current Kotlin block or type. |
| 39 | `if (state.isLoading) {` | Shows the progress section only while state.isLoading is true. |
| 40 | `Row(horizontalArrangement = Arrangement.spacedBy(12.dp), verticalAlignment = Alignment.CenterVertically) {` | Places progress indicator and status message next to one another. |
| 41 | `CircularProgressIndicator(Modifier.size(24.dp), color = PrimaryGreen)` | Shows an indeterminate progress indicator rather than inventing an upload percentage. |
| 42 | `Text("Sending to the test server…", color = PrimaryTextColor)` | Displays a pending-network message. |
| 43 | `}` | Closes the current Kotlin block or type. |
| 44 | `Text("The test server may take a moment to wake up.", fontSize = 13.sp, color = SecondaryTextColor)` | Explains that the test deployment can take time to become responsive. |
| 45 | `}` | Closes the current Kotlin block or type. |
| 46 | `state.receipt?.let { receipt ->` | Only renders the successful server card when a receipt exists. |
| 47 | `Surface(shape = RoundedCornerShape(14.dp), color = MaterialTheme.colorScheme.surface) {` | Uses a rounded surface for the server response bubble. |
| 48 | `Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {` | Applies padding and spacing inside the server card. |
| 49 | `Text("Server · Upload confirmed", fontWeight = FontWeight.SemiBold, color = PrimaryGreen)` | Labels the response as upload confirmation, not an ML verdict. |
| 50 | `Text(receipt.message, color = PrimaryTextColor)` | Shows the actual server message from the JSON response. |
| 51 | `Text("Received ${receipt.width} × ${receipt.height} image · ${receipt.sizeBytes} bytes", fontSize = 13.sp)` | Shows server-reported dimensions and received bytes to help check the upload. |
| 52 | `Text("Experiment: ${receipt.experimentId}\nRequest ID: ${receipt.requestId}", fontSize = 12.sp, color = SecondaryTextColor)` | Displays the echoed experiment ID and unique request ID for debugging. |
| 53 | `Text("SHA-256: ${receipt.sha256}", fontSize = 11.sp, color = SecondaryTextColor)` | Displays the server SHA-256. The UI does not independently verify the hash. |
| 54 | `}` | Closes the current Kotlin block or type. |
| 55 | `}` | Closes the current Kotlin block or type. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 56 | `}` | Closes the current Kotlin block or type. |
| 57 | `state.error?.let {` | Only renders the error/retry section when state.error is non-null. |
| 58 | `Text(it, color = MaterialTheme.colorScheme.error)` | Displays the failure in the Material error color. |
| 59 | `Button(onClick = onRetry, colors = ButtonDefaults.buttonColors(containerColor = PrimaryGreen)) { Text("Retry upload") }` | Calls the supplied retry callback when Retry upload is tapped. |
| 60 | `}` | Closes the current Kotlin block or type. |
| 61 | `if (state.imageUri == null) {` | Recognizes an empty state, such as process recreation without a retained draft. |
| 62 | `Text("This test conversation cannot be restored. Return Home and submit the image again.", color = SecondaryTextColor)` | Explains that this deployment has no session-restore operation; the user must return Home and resubmit. |
| 63 | `}` | Closes the current Kotlin block or type. |
| 64 | `Text("Follow-up messaging will be available when the conversation API is ready.", color = SecondaryTextColor, fontSize = 13.sp)` | States that follow-up messages are unavailable; no fake chat replies are generated. |
| 65 | `}` | Closes the current Kotlin block or type. |
| 66 | `}` | Closes the current Kotlin block or type. |

# 8. MainActivity.kt

app/src/main/java/com/example/gitgood/MainActivity.kt

Modified for API integration. Full file shown. Existing auth/experiment navigation is included as context; the API additions are ConversationViewModel, Conversation entry, Home Continue, and exit/Back cleanup.

Snapshot SHA-256: c58a3c32415602ac63b690cddcc630475ceb178218d73311eb653a697da5c024

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood` | Places this file in the com.example.gitgood Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import android.os.Bundle` | Imports Activity saved-state parameters. |
| 4 | `import androidx.activity.ComponentActivity` | Imports the Activity base class used by Compose. |
| 5 | `import androidx.activity.compose.setContent` | Imports installing the Compose UI tree. |
| 6 | `import androidx.activity.enableEdgeToEdge` | Imports edge-to-edge window configuration. |
| 7 | `import androidx.activity.viewModels` | Imports Activity-scoped ViewModel delegates. |
| 8 | `import androidx.compose.foundation.layout.fillMaxSize` | Imports filling the available layout area. |
| 9 | `import androidx.compose.foundation.layout.padding` | Imports layout padding modifiers. |
| 10 | `import androidx.compose.material3.Scaffold` | Imports screen structure and content insets. |
| 11 | `import androidx.compose.runtime.LaunchedEffect` | Imports effects tied to a Compose key/lifetime. |
| 12 | `import androidx.compose.runtime.getValue` | Imports delegated Compose state reads using by. |
| 13 | `import androidx.compose.ui.Modifier` | Imports layout, appearance and behavior modifiers. |
| 14 | `import androidx.compose.ui.platform.LocalContext` | Imports the current UI Context. |
| 15 | `import androidx.lifecycle.compose.collectAsStateWithLifecycle` | Imports lifecycle-aware collection of state streams in Compose. |
| 16 | `import androidx.lifecycle.viewmodel.navigation3.rememberViewModelStoreNavEntryDecorator` | Imports per-navigation-entry ViewModel-store support. |
| 17 | `import androidx.navigation3.runtime.entryProvider` | Imports typed navigation entry declarations. |
| 18 | `import androidx.navigation3.runtime.rememberNavBackStack` | Imports saved typed back-stack state. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 19 | `import androidx.navigation3.runtime.rememberSaveableStateHolderNavEntryDecorator` | Imports saveable state attached to navigation entries. |
| 20 | `import androidx.navigation3.ui.NavDisplay` | Imports rendering the current Navigation 3 destinations. |
| 21 | `import com.example.gitgood.navigation.HomeDestination` | Imports HomeDestination so this file can use that existing app class or composable by its short name. |
| 22 | `import com.example.gitgood.navigation.RegisterDestination` | Imports RegisterDestination so this file can use that existing app class or composable by its short name. |
| 23 | `import com.example.gitgood.navigation.ScreenRoute` | Imports ScreenRoute so this file can use that existing app class or composable by its short name. |
| 24 | `import com.example.gitgood.ui.screen.auth.login.LoginScreenLayout` | Imports LoginScreenLayout so this file can use that existing app class or composable by its short name. |
| 25 | `import com.example.gitgood.ui.theme.GitGoodTheme` | Imports GitGoodTheme so this file can use that existing app class or composable by its short name. |
| 26 | `import com.example.gitgood.ui.theme.screen.CircuitlySplashScreen` | Imports CircuitlySplashScreen so this file can use that existing app class or composable by its short name. |
| 27 | `import com.example.gitgood.ui.theme.screen.auth.loading.CircuitlyWelcomeScreenLayout` | Imports CircuitlyWelcomeScreenLayout so this file can use that existing app class or composable by its short name. |
| 28 | `import com.example.gitgood.viewModel.ExperimentViewModel` | Imports ExperimentViewModel so this file can use that existing app class or composable by its short name. |
| 29 | `import com.example.gitgood.ui.theme.screen.experiment.ExperimentScreen` | Imports ExperimentScreen so this file can use that existing app class or composable by its short name. |
| 30 | `import com.example.gitgood.viewModel.ConversationViewModel` | Imports ConversationViewModel so this file can use that existing app class or composable by its short name. |
| 31 | `import com.example.gitgood.ui.theme.screen.conversation.ConversationScreen` | Imports ConversationScreen so this file can use that existing app class or composable by its short name. |
| 32 | `import com.example.gitgood.viewModel.HomeViewModel` | Imports HomeViewModel so this file can use that existing app class or composable by its short name. |
| 33 | `import com.example.gitgood.viewModel.LoginViewModel` | Imports LoginViewModel so this file can use that existing app class or composable by its short name. |
| 34 | `import dagger.hilt.android.AndroidEntryPoint` | Imports Hilt integration for the Activity. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 35 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 36 | `@AndroidEntryPoint` | Allows this Activity to obtain Hilt-injected ViewModels. |
| 37 | `class MainActivity : ComponentActivity() {` | Defines the app Android entry Activity. |
| 38 | `private val experimentViewModel: ExperimentViewModel by viewModels()` | Keeps selected-experiment state in an Activity-scoped ViewModel. |
| 39 | `private val conversationViewModel: ConversationViewModel by viewModels()` | Adds the Activity-scoped ConversationViewModel used by Continue, response UI, Retry and Exit. |
| 40 | `private val homeViewModel: HomeViewModel by viewModels()` | Obtains shared Home state containing the image URI and description. |
| 41 | `private val loginViewModel: LoginViewModel by viewModels()` | Obtains the existing authentication ViewModel. |
| 42 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 43 | `override fun onCreate(savedInstanceState: Bundle?) {` | Receives the Android Activity creation callback. |
| 44 | `super.onCreate(savedInstanceState)` | Initializes the ComponentActivity superclass before composing UI. |
| 45 | `enableEdgeToEdge()` | Enables drawing behind system bars; Scaffold padding is applied below. |
| 46 | `setContent {` | Starts the Compose content tree. |
| 47 | `GitGoodTheme {` | Applies the existing app colors and typography. |
| 48 | `val user by loginViewModel.currentUser.collectAsStateWithLifecycle()` | Observes Firebase user/session state with lifecycle-aware Compose collection. |
| 49 | `val loginState by loginViewModel.loginUiState.collectAsStateWithLifecycle()` | Observes login form/error state without directly accessing Firebase from UI widgets. |
| 50 | `val context = LocalContext.current` | Obtains the current UI Context for Google account-picker actions. |
| 51 | `val backStack = rememberNavBackStack(ScreenRoute.Auth.Splash)` | Creates the saved typed Navigation 3 back stack starting at Splash. |
| 52 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 53 | `LaunchedEffect(user?.uid) {` | Runs session-based navigation only when the user UID key changes or the effect first enters composition. |
| 54 | `if (user == null) {` | Handles the signed-out state. |
| 55 | `conversationViewModel.exit()` | New API cleanup: cancels local upload work and clears conversation data when signed out. |
| 56 | `homeViewModel.reset()` | Clears image and description from Home state. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 57 | `experimentViewModel.clear()` | Clears the selected experiment for the signed-out user. |
| 58 | `if (backStack.lastOrNull() == ScreenRoute.Auth.Splash \|\|` | Starts checking whether the current stack needs a signed-out redirect. |
| 59 | `backStack.any { it == ScreenRoute.Home \|\| it == ScreenRoute.Experiments \|\| it == ScreenRoute.Conversation }) {` | Includes Conversation among authenticated destinations that must not remain visible after sign-out. |
| 60 | `backStack.clear()` | Removes authenticated navigation history. |
| 61 | `backStack.add(ScreenRoute.Auth.Landing)` | Makes the welcome/landing screen the signed-out root. |
| 62 | `}` | Closes the current Kotlin block or type. |
| 63 | `} else if (backStack.any { it is ScreenRoute.Auth } \|\|` | Handles authenticated users still on an auth route, or an invalid restored Home destination. |
| 64 | `(backStack.lastOrNull() == ScreenRoute.Home && experimentViewModel.selectedExperiment == null)) {` | Detects Home without a selected experiment and routes back to selection. |
| 65 | `backStack.clear()` | Clears the old authentication or invalid navigation stack. |
| 66 | `backStack.add(ScreenRoute.Experiments)` | Uses experiment selection as the authenticated starting destination. |
| 67 | `}` | Closes the current Kotlin block or type. |
| 68 | `}` | Closes the current Kotlin block or type. |
| 69 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 70 | `Scaffold { padding ->` | Provides Scaffold content padding for system UI/layout insets. |
| 71 | `NavDisplay(` | Starts Navigation 3 destination rendering. |
| 72 | `backStack = backStack,` | Supplies the actual list of navigation keys. |
| 73 | `modifier = Modifier.fillMaxSize().padding(padding),` | Fills the content area and applies Scaffold padding. |
| 74 | `onBack = {` | Defines explicit system-Back behavior. |
| 75 | `if (backStack.size > 1) {` | Only pops when at least one destination remains underneath. |
| 76 | `if (backStack.lastOrNull() == ScreenRoute.Conversation) {` | New API behavior: recognizes Back from Conversation. |
| 77 | `conversationViewModel.exit()` | Cancels pending conversation work and clears local response state. |
| 78 | `homeViewModel.reset()` | Clears the previous Home image/description on conversation exit. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 79 | `}` | Closes the current Kotlin block or type. |
| 80 | `loginViewModel.clearMessage()` | Clears old authentication messages before leaving the current route. |
| 81 | `backStack.removeAt(backStack.lastIndex)` | Removes the current top route; from Conversation this reveals Home. |
| 82 | `}` | Closes the current Kotlin block or type. |
| 83 | `},` | Closes this callback/block and separates it from the next argument. |
| 84 | `entryDecorators = listOf(` | Installs decorators supporting saved destination state and ViewModel lifetimes. |
| 85 | `rememberSaveableStateHolderNavEntryDecorator(),` | Preserves rememberSaveable state attached to each navigation entry. |
| 86 | `rememberViewModelStoreNavEntryDecorator()` | Provides entry ViewModel stores; the auth/upload ViewModels used here remain Activity-scoped. |
| 87 | `),` | Closes this call and separates it from the next argument. |
| 88 | `entryProvider = entryProvider {` | Maps typed route keys to screen content. |
| 89 | `entry<ScreenRoute.Auth.Splash> { CircuitlySplashScreen() }` | Keeps the existing Splash route and screen. |
| 90 | `entry<ScreenRoute.Auth.Landing> {` | Keeps the existing welcome/landing route. |
| 91 | `CircuitlyWelcomeScreenLayout(` | Renders the welcome screen. |
| 92 | `onGetStartedClick = { backStack.add(ScreenRoute.Auth.Register) },` | Pushes Register when Get started is tapped. |
| 93 | `onLoginClick = { backStack.add(ScreenRoute.Auth.Login) }` | Pushes Login when the landing Login action is tapped. |
| 94 | `)` | Closes the current argument or property list. |
| 95 | `}` | Closes the current Kotlin block or type. |
| 96 | `entry<ScreenRoute.Auth.Login> {` | Keeps the existing Login route. |
| 97 | `LoginScreenLayout(` | Renders the login UI using state and callbacks. |
| 98 | `uiState = loginState,` | Supplies the observed login form state. |
| 99 | `onEmailChange = loginViewModel::onEmailChange,` | Sends email edits to the login ViewModel. |
| 100 | `onPasswordChange = loginViewModel::onPasswordChange,` | Sends password edits to the login ViewModel. |
| 101 | `onPasswordVisibilityToggle = loginViewModel::togglePasswordVisibility,` | Delegates the password-visibility toggle. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 102 | `onLoginClick = loginViewModel::onLoginClick,` | Delegates email/password login. |
| 103 | `onForgotPasswordClick = loginViewModel::onForgotPasswordClick,` | Delegates password-reset email requests. |
| 104 | `onSignUpClick = {` | Begins switching from Login to Register. |
| 105 | `loginViewModel.clearMessage()` | Clears old login messages. |
| 106 | `backStack.removeAt(backStack.lastIndex)` | Removes the current Login entry to avoid duplicate form entries. |
| 107 | `backStack.add(ScreenRoute.Auth.Register)` | Adds the Register route. |
| 108 | `},` | Closes this callback/block and separates it from the next argument. |
| 109 | `onBackClick = {` | Defines the login layout Back callback. |
| 110 | `if (backStack.size > 1) backStack.removeAt(backStack.lastIndex)` | Pops the login route when another entry remains. |
| 111 | `},` | Closes this callback/block and separates it from the next argument. |
| 112 | `onGoogleClick = { loginViewModel.onGoogleClick(context) }` | Launches existing Google authentication with the UI Context. |
| 113 | `)` | Closes the current argument or property list. |
| 114 | `}` | Closes the current Kotlin block or type. |
| 115 | `entry<ScreenRoute.Auth.Register> {` | Keeps the Register route. |
| 116 | `RegisterDestination(loginViewModel, onLogin = {` | Renders the registration wrapper with a navigation callback. |
| 117 | `loginViewModel.clearMessage()` | Clears old messages when switching to Login. |
| 118 | `backStack.removeAt(backStack.lastIndex)` | Replaces the registration route rather than stacking repeated form switches. |
| 119 | `backStack.add(ScreenRoute.Auth.Login)` | Adds the Login route. |
| 120 | `})` | Closes the lambda and its surrounding call. |
| 121 | `}` | Closes the current Kotlin block or type. |
| 122 | `entry<ScreenRoute.Experiments> {` | Keeps the experiment-selection route created before API integration. |
| 123 | `val selectedId by experimentViewModel.selectedId.collectAsStateWithLifecycle()` | Observes the selected experiment ID for selection highlighting. |
| 124 | `ExperimentScreen(` | Renders the experiment list. |
| 125 | `experiments = experimentViewModel.experiments,` | Supplies the local catalog of five experiments. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 126 | `selectedId = selectedId,` | Supplies the selected radio-card ID. |
| 127 | `onSelect = { id ->` | Handles a selection change. |
| 128 | `if (id != experimentViewModel.selectedId.value) homeViewModel.reset()` | Clears a previous circuit draft when selecting a different experiment. |
| 129 | `experimentViewModel.select(id)` | Saves the new experiment ID. |
| 130 | `},` | Closes this callback/block and separates it from the next argument. |
| 131 | `onContinue = {` | Defines Continue from experiment selection, which is separate from upload Continue on Home. |
| 132 | `if (experimentViewModel.selectedExperiment != null && backStack.lastOrNull() != ScreenRoute.Home) {` | Requires a selected experiment and avoids pushing a duplicate Home route. |
| 133 | `backStack.add(ScreenRoute.Home)` | Opens Home so the user can choose/capture a circuit image. |
| 134 | `}` | Closes the current Kotlin block or type. |
| 135 | `}` | Closes the current Kotlin block or type. |
| 136 | `)` | Closes the current argument or property list. |
| 137 | `}` | Closes the current Kotlin block or type. |
| 138 | `entry<ScreenRoute.Conversation> {` | New route entry for the upload-test conversation. |
| 139 | `val conversation by conversationViewModel.uiState.collectAsStateWithLifecycle()` | Observes loading, receipt, error and submission state from ConversationViewModel. |
| 140 | `ConversationScreen(` | Renders the conversation-style test response screen. |
| 141 | `state = conversation,` | Supplies the latest observed conversation state. |
| 142 | `onRetry = conversationViewModel::retry,` | Connects Retry upload to the ViewModel retry operation. |
| 143 | `onExit = {` | Defines the visible Exit conversation action. |
| 144 | `conversationViewModel.exit()` | Cancels/clears the local upload conversation. |
| 145 | `homeViewModel.reset()` | Clears image and description while keeping experiment selection. |
| 146 | `if (backStack.lastOrNull() == ScreenRoute.Conversation) {` | Checks that Conversation is still the current route before removing it. |
| 147 | `backStack.removeAt(backStack.lastIndex)` | Pops Conversation so Home becomes visible again. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 148 | `}` | Closes the current Kotlin block or type. |
| 149 | `}` | Closes the current Kotlin block or type. |
| 150 | `)` | Closes the current argument or property list. |
| 151 | `}` | Closes the current Kotlin block or type. |
| 152 | `entry<ScreenRoute.Home> {` | Renders the existing Home route. |
| 153 | `HomeDestination(loginViewModel, homeViewModel,` | Passes authentication and Home state through the Home destination wrapper. |
| 154 | `experimentName = experimentViewModel.selectedExperiment?.name,` | Displays the selected experiment name on Home. |
| 155 | `onContinue = {` | New callback: performs the Home Continue submission. |
| 156 | `val experiment = experimentViewModel.selectedExperiment` | Reads the currently selected experiment object. |
| 157 | `val draft = homeViewModel.uiState.value` | Captures the Home form state for this submission. |
| 158 | `val image = draft.selectedImageUri` | Reads the selected image URI, which may come from picker or camera. |
| 159 | `if (experiment != null && image != null && backStack.lastOrNull() == ScreenRoute.Home) {` | Requires experiment and image and guards against another route transition after leaving Home. |
| 160 | `conversationViewModel.start(experiment.id, experiment.name, image, draft.description)` | Starts the real upload using experiment ID/name, image URI and description. |
| 161 | `backStack.add(ScreenRoute.Conversation)` | Pushes Conversation immediately so its loading state is visible. |
| 162 | `}` | Closes the current Kotlin block or type. |
| 163 | `},` | Closes this callback/block and separates it from the next argument. |
| 164 | `onChangeExperiment = {` | Preserves the Change experiment callback. |
| 165 | `if (backStack.size > 1) backStack.removeAt(backStack.lastIndex)` | Returns to selection by popping Home if another entry exists. |
| 166 | `}` | Closes the current Kotlin block or type. |
| 167 | `)` | Closes the current argument or property list. |
| 168 | `}` | Closes the current Kotlin block or type. |
| 169 | `}` | Closes the current Kotlin block or type. |
| 170 | `)` | Closes the current argument or property list. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 171 | } | Closes the current Kotlin block or type. |
| 172 | } | Closes the current Kotlin block or type. |
| 173 | } | Closes the current Kotlin block or type. |
| 174 | } | Closes the current Kotlin block or type. |
| 175 | } | Closes the current Kotlin block or type. |

# 9. ScreenRoute.kt

app/src/main/java/com/example/gitgood/navigation/ScreenRoute.kt

Modified for API integration. The Conversation route was added. Existing routes remain available.

Snapshot SHA-256: fceb95f7b2b678922487cfb46a9639b9a7a2ee8a97ed976ede1504e9c7d44ef5

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.navigation` | Places this file in the com.example.gitgood.navigation Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import androidx.navigation3.runtime.NavKey` | Imports typed Navigation 3 back-stack entries. |
| 4 | `import kotlinx.serialization.Serializable` | Imports serialization support for saved route keys. |
| 5 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 6 | `@Serializable` | Enables Kotlin serialization for the route hierarchy. |
| 7 | `sealed interface ScreenRoute : NavKey {` | Defines the closed set of typed Navigation 3 keys. |
| 8 | `@Serializable` | Makes the nested authentication route hierarchy serializable. |
| 9 | `sealed interface Auth : ScreenRoute {` | Groups authentication routes under ScreenRoute.Auth. |
| 10 | `@Serializable data object Splash : Auth` | Defines the existing startup route. |
| 11 | `@Serializable data object Landing : Auth` | Defines the welcome route. |
| 12 | `@Serializable data object Login : Auth` | Defines the login route. |
| 13 | `@Serializable data object Register : Auth` | Defines the registration route. |
| 14 | `}` | Closes the current Kotlin block or type. |
| 15 | `@Serializable data object Experiments : ScreenRoute` | Defines the experiment-selection route. |
| 16 | `@Serializable data object Conversation : ScreenRoute` | New API destination: Conversation identifies the local upload-test screen. It carries no backend session ID. |
| 17 | `@Serializable data object Home : ScreenRoute` | Defines the image-and-description Home route. |
| 18 | `}` | Closes the current Kotlin block or type. |

# 10. AppNavigation.kt

app/src/main/java/com/example/gitgood/navigation/AppNavigation.kt

Modified for API integration. Full file shown, including prior registration, picker and camera code. The API-specific change receives and forwards onContinue.

Snapshot SHA-256: 701cf1d68d66127ec7114f8ba5bda0dc132d5549d36be9d0e9b290bf11303eaf

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.navigation` | Places this file in the com.example.gitgood.navigation Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import android.content.ActivityNotFoundException` | Imports camera launch failure handling. |
| 4 | `import androidx.core.content.FileProvider` | Imports content URIs for private camera output files. |
| 5 | `import java.io.File` | Imports temporary camera files. |
| 6 | `import java.io.IOException` | Imports readable file/network failure exceptions. |
| 7 | `import androidx.activity.compose.rememberLauncherForActivityResult` | Imports photo-picker and camera result registration. |
| 8 | `import androidx.activity.result.PickVisualMediaRequest` | Imports image-only picker requests. |
| 9 | `import androidx.activity.result.contract.ActivityResultContracts` | Imports typed camera/picker Android contracts. |
| 10 | `import com.example.gitgood.viewModel.HomeViewModel` | Imports HomeViewModel so this file can use that existing app class or composable by its short name. |
| 11 | `import androidx.compose.foundation.layout.Column` | Imports Column so this file can use that existing app class or composable by its short name. |
| 12 | `import androidx.compose.foundation.layout.padding` | Imports layout padding modifiers. |
| 13 | `import androidx.compose.material3.MaterialTheme` | Imports MaterialTheme so this file can use that existing app class or composable by its short name. |
| 14 | `import androidx.compose.material3.Text` | Imports Text so this file can use that existing app class or composable by its short name. |
| 15 | `import androidx.compose.runtime.*` | Imports the members of androidx.compose.runtime used by the UI or state code in this file. |
| 16 | `import androidx.compose.runtime.saveable.rememberSaveable` | Imports UI values restored across activity recreation. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 17 | `import androidx.compose.ui.Modifier` | Imports layout, appearance and behavior modifiers. |
| 18 | `import androidx.compose.ui.platform.LocalContext` | Imports the current UI Context. |
| 19 | `import androidx.compose.ui.unit.dp` | Imports density-independent layout dimensions. |
| 20 | `import androidx.lifecycle.compose.collectAsStateWithLifecycle` | Imports lifecycle-aware collection of state streams in Compose. |
| 21 | `import com.example.gitgood.ui.screen.auth.register.RegisterScreenLayout` | Imports RegisterScreenLayout so this file can use that existing app class or composable by its short name. |
| 22 | `import com.example.gitgood.ui.screen.auth.register.RegisterUiState` | Imports RegisterUiState so this file can use that existing app class or composable by its short name. |
| 23 | `import com.example.gitgood.ui.theme.screen.home.HomeScreen` | Imports HomeScreen so this file can use that existing app class or composable by its short name. |
| 24 | `import com.example.gitgood.viewModel.LoginViewModel` | Imports LoginViewModel so this file can use that existing app class or composable by its short name. |
| 25 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 26 | `@Composable` | Marks the next function as Compose UI content. |
| 27 | `fun RegisterDestination(viewModel: LoginViewModel, onLogin: () -> Unit) {` | Existing registration wrapper, included for complete file context; it is not part of the HTTP upload logic. |
| 28 | `val state by viewModel.loginUiState.collectAsStateWithLifecycle()` | Collects login/registration operation state. |
| 29 | `val context = LocalContext.current` | Gets the UI Context for Google sign-in. |
| 30 | `var email by rememberSaveable { mutableStateOf("") }` | Keeps registration email in saveable local UI state. |
| 31 | `// Passwords remain in memory and are discarded when this destination is removed.` | Explains why registration passwords are not intentionally stored in saved state. |
| 32 | `var password by remember { mutableStateOf("") }` | Keeps the registration password in ordinary remembered memory. |
| 33 | `var confirmation by remember { mutableStateOf("") }` | Keeps password confirmation in memory. |
| 34 | `var showPassword by remember { mutableStateOf(false) }` | Tracks registration password visibility. |
| 35 | `var showConfirmation by remember { mutableStateOf(false) }` | Tracks confirmation visibility. |
| 36 | `Column {` | Stacks registration errors and form content vertically. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 37 | `state.errorMessage?.let {` | Only displays an authentication error when one exists. |
| 38 | `Text(it, color = MaterialTheme.colorScheme.error, modifier = Modifier.padding(16.dp))` | Shows the error with spacing and the theme error color. |
| 39 | `}` | Closes the current Kotlin block or type. |
| 40 | `RegisterScreenLayout(` | Calls the existing registration layout. |
| 41 | `uiState = RegisterUiState(email, password, confirmation, showPassword, showConfirmation, state.isLoading),` | Builds the registration UI data from local inputs and shared loading state. |
| 42 | `onEmailInput = { email = it },` | Updates registration email. |
| 43 | `onPasswordInput = { password = it },` | Updates registration password. |
| 44 | `onConfirmPasswordInput = { confirmation = it },` | Updates confirmation text. |
| 45 | `onPasswordVisibilityToggle = { showPassword = !showPassword },` | Toggles password visibility. |
| 46 | `onConfirmPasswordVisibilityToggle = { showConfirmation = !showConfirmation },` | Toggles confirmation visibility. |
| 47 | `onSignUpClick = { viewModel.onSignUpClick(email, password, confirmation) },` | Submits account registration to the existing login/auth ViewModel. |
| 48 | `onGoogleClick = { viewModel.onGoogleClick(context) },` | Starts the existing Google authentication flow. |
| 49 | `onLoginClick = onLogin` | Delegates navigation back to Login. |
| 50 | `)` | Closes the current argument or property list. |
| 51 | `}` | Closes the current Kotlin block or type. |
| 52 | `}` | Closes the current Kotlin block or type. |
| 53 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 54 | `@Composable` | Marks the next function as Compose UI content. |
| 55 | `fun HomeDestination(` | Defines the Home destination wrapper where picker/camera launchers live. |
| 56 | `viewModel: LoginViewModel,` | Receives the existing LoginViewModel for its error state. |
| 57 | `homeViewModel: HomeViewModel,` | Receives the HomeViewModel containing image/description state. |
| 58 | `experimentName: String? = null,` | Receives the selected experiment name for display. |
| 59 | `onChangeExperiment: () -> Unit = {},` | Receives the callback that returns to experiment selection. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 60 | `onContinue: () -> Unit = {}` | New API integration parameter: accepts the Home Continue action from MainActivity. Default is a no-op for optional callers. |
| 61 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 62 | `val state by viewModel.loginUiState.collectAsStateWithLifecycle()` | Observes the existing authentication error state. |
| 63 | `val homeState by homeViewModel.uiState.collectAsStateWithLifecycle()` | Observes selected image URI and description from HomeViewModel. |
| 64 | `val context = LocalContext.current` | Obtains Context for image FileProvider operations. |
| 65 | `// Save the destination across activity recreation while the camera is open.` | Documents that the pending camera path must survive Activity recreation while the camera is open. |
| 66 | `var pendingPhotoPath by rememberSaveable { mutableStateOf<String?>(null) }` | Saves the pending camera-output path in destination saveable state. |
| 67 | `var cameraError by rememberSaveable { mutableStateOf<String?>(null) }` | Keeps a local camera failure message. |
| 68 | `val camera = rememberLauncherForActivityResult(ActivityResultContracts.TakePicture()) { success ->` | Registers the TakePicture activity-result callback. |
| 69 | `val photo = pendingPhotoPath?.let(::File)` | Reconstructs the pending file from its saved path. |
| 70 | `pendingPhotoPath = null` | Clears the pending marker after the camera returns. |
| 71 | `if (success && photo != null && photo.length() > 0L) {` | Accepts only a reported-success capture with a nonempty output file. |
| 72 | `homeViewModel.selectImage(` | Starts updating the shared selected-image state. |
| 73 | `FileProvider.getUriForFile(context, "${context.packageName}.camera.files", photo)` | Converts the captured private file into a shareable content URI. |
| 74 | `)` | Closes the current argument or property list. |
| 75 | `cameraError = null` | Clears an earlier camera error after a successful capture. |
| 76 | `} else {` | Handles cancellation or an unsuccessful capture. |
| 77 | `photo?.delete()` | Deletes only the unused temporary capture file. |
| 78 | `if (success) cameraError = "The camera could not save the photo. Please try again."` | Reports a missing/empty output when the camera claimed success. |
| 79 | `}` | Closes the current Kotlin block or type. |
| 80 | `}` | Closes the current Kotlin block or type. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 81 | `val picker = rememberLauncherForActivityResult(ActivityResultContracts.PickVisualMedia()) { uri ->` | Registers the existing single-image photo picker. |
| 82 | `if (uri != null) homeViewModel.selectImage(uri)` | Updates the shared selected-image state only when a URI was returned; cancellation keeps the prior image. |
| 83 | `}` | Closes the current Kotlin block or type. |
| 84 | `HomeScreen(` | Calls the stateless Home layout with current values and callbacks. |
| 85 | `experimentName = experimentName,` | Passes the selected experiment label. |
| 86 | `onChangeExperiment = onChangeExperiment,` | Passes experiment-change navigation. |
| 87 | `selectedImageUri = homeState.selectedImageUri,` | Passes the local URI used by preview and upload. |
| 88 | `description = homeState.description,` | Passes the description that will be included in the API request. |
| 89 | `onDescriptionChange = homeViewModel::onDescriptionChange,` | Sends description edits into HomeViewModel. |
| 90 | `onRemoveImageClick = homeViewModel::removeImage,` | Connects Remove image to clearing the selection. |
| 91 | `onUploadClick = {` | Defines the picker-launch callback. |
| 92 | `picker.launch(PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly))` | Launches the picker restricted to images. |
| 93 | `},` | Closes this callback/block and separates it from the next argument. |
| 94 | `onTakePhotoClick = {` | Defines the camera-launch callback, retained from earlier camera work. |
| 95 | `if (pendingPhotoPath == null) {` | Avoids launching another camera request while one has a pending file. |
| 96 | `cameraError = null` | Clears a previous camera launch error. |
| 97 | `try {` | Catches file preparation and camera launch failures. |
| 98 | `val directory = File(context.cacheDir, "camera")` | Uses the app-private camera cache directory. |
| 99 | `if (!directory.exists() && !directory.mkdirs()) {` | Creates the directory if necessary and checks failure. |
| 100 | `throw IOException("Could not create photo directory")` | Turns directory creation failure into an exception handled below. |
| 101 | `}` | Closes the current Kotlin block or type. |
| 102 | `val photo = File.createTempFile("capture_", ".jpg", directory)` | Creates the temporary JPEG capture destination. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 103 | `pendingPhotoPath = photo.absolutePath` | Retains its path for the activity result. |
| 104 | `val uri = FileProvider.getUriForFile(` | Starts constructing the camera content URI. |
| 105 | `context, "${context.packageName}.camera.files", photo` | Uses the provider authority registered in the manifest. |
| 106 | `)` | Closes the current argument or property list. |
| 107 | `camera.launch(uri)` | Opens the external camera with the destination URI. |
| 108 | `} catch (_: ActivityNotFoundException) {` | Handles the absence of a compatible camera application. |
| 109 | `pendingPhotoPath?.let { File(it).delete() }` | Deletes the unused pending file after launch failure. |
| 110 | `pendingPhotoPath = null` | Clears the pending marker. |
| 111 | `cameraError = "No camera app is available on this device."` | Shows a readable no-camera-app error. |
| 112 | `} catch (_: Exception) {` | Handles other file or camera-launch failures. |
| 113 | `pendingPhotoPath?.let { File(it).delete() }` | Deletes an unused temporary file after the failure. |
| 114 | `pendingPhotoPath = null` | Clears the pending marker. |
| 115 | `cameraError = "Could not open the camera. Please try again."` | Shows a generic camera-launch error. |
| 116 | `}` | Closes the current Kotlin block or type. |
| 117 | `}` | Closes the current Kotlin block or type. |
| 118 | `},` | Closes this callback/block and separates it from the next argument. |
| 119 | `onContinueClick = onContinue,` | New API wiring: forwards HomeScreen Continue to the callback supplied by MainActivity instead of an empty action. |
| 120 | `errorMessage = cameraError ?: state.errorMessage` | Displays a camera error first, otherwise an existing authentication error. Upload errors appear on ConversationScreen. |
| 121 | `)` | Closes the current argument or property list. |
| 122 | `}` | Closes the current Kotlin block or type. |

# 11. HomeScreen.kt

app/src/main/java/com/example/gitgood/ui/theme/screen/home/HomeScreen.kt

Modified for API integration. Full layout shown for context. The API change adds description counter/error state and requires image plus valid description to enable Continue.

Snapshot SHA-256: 8c180d77de82f1642046547a515be224f4f28db7f7dc33ef5819458aa3e91bef

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.ui.theme.screen.home` | Places this file in the com.example.gitgood.ui.theme.screen.home Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import android.net.Uri` | Imports Android identifier for a selected or captured image. |
| 4 | `import androidx.compose.ui.layout.ContentScale` | Imports how an image is fitted to its display bounds. |
| 5 | `import coil3.compose.AsyncImage` | Imports Coil image rendering from a content URI. |
| 6 | `import androidx.compose.foundation.BorderStroke` | Imports borders for cards, fields and outlined buttons. |
| 7 | `import androidx.compose.foundation.background` | Imports background color modifiers. |
| 8 | `import androidx.compose.foundation.layout.*` | Imports the members of androidx.compose.foundation.layout used by the UI or state code in this file. |
| 9 | `import androidx.compose.foundation.rememberScrollState` | Imports remembered scroll offsets. |
| 10 | `import androidx.compose.foundation.shape.RoundedCornerShape` | Imports rounded cards/buttons/input fields. |
| 11 | `import androidx.compose.foundation.verticalScroll` | Imports scrollable column behavior. |
| 12 | `import androidx.compose.material.icons.Icons` | Imports the Compose icon collection. |
| 13 | `import androidx.compose.material.icons.outlined.AddPhotoAlternate` | Imports the upload-image icon. |
| 14 | `import androidx.compose.material.icons.outlined.Image` | Imports the empty-preview image icon. |
| 15 | `import androidx.compose.material.icons.outlined.PhotoCamera` | Imports the Take photo icon. |
| 16 | `import androidx.compose.material3.*` | Imports the members of androidx.compose.material3 used by the UI or state code in this file. |
| 17 | `import androidx.compose.runtime.Composable` | Imports Compose UI function annotation. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 18 | `import androidx.compose.ui.Alignment` | Imports alignment options for Compose layouts. |
| 19 | `import androidx.compose.ui.Modifier` | Imports layout, appearance and behavior modifiers. |
| 20 | `import androidx.compose.ui.graphics.Color` | Imports explicit color values such as white. |
| 21 | `import androidx.compose.ui.tooling.preview.Preview` | Imports Android Studio design-time preview. |
| 22 | `import androidx.compose.ui.text.font.FontWeight` | Imports text emphasis options. |
| 23 | `import androidx.compose.ui.unit.sp` | Imports font-size dimensions. |
| 24 | `import androidx.compose.ui.unit.dp` | Imports density-independent layout dimensions. |
| 25 | `import com.example.gitgood.ui.theme.*` | Imports the members of com.example.gitgood.ui.theme used by the UI or state code in this file. |
| 26 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 27 | `@Composable` | Marks the next function as Compose UI content. |
| 28 | `fun HomeScreen(` | Defines the stateless Home composable; network work remains outside this screen. |
| 29 | `onUploadClick: () -> Unit,` | Receives the photo-picker launch callback. |
| 30 | `onTakePhotoClick: () -> Unit,` | Receives the camera-launch callback. |
| 31 | `onContinueClick: () -> Unit,` | Receives the Continue submission callback added to actual navigation wiring. |
| 32 | `experimentName: String? = null,` | Receives the selected experiment display name. |
| 33 | `onChangeExperiment: () -> Unit = {},` | Receives Change experiment navigation. |
| 34 | `selectedImageUri: Uri? = null,` | Receives the image URI from HomeViewModel. |
| 35 | `description: String = "",` | Receives the current description text. |
| 36 | `onDescriptionChange: (String) -> Unit = {},` | Receives a callback to update description state. |
| 37 | `onRemoveImageClick: () -> Unit = {},` | Receives the selected-image removal callback. |
| 38 | `modifier: Modifier = Modifier,` | Allows callers to add layout modifiers. |
| 39 | `errorMessage: String? = null` | Receives camera/auth errors displayed on Home, separate from upload errors. |
| 40 | `) {` | Finishes the argument list and starts its function or UI-content block. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 41 | `Column(` | Begins the vertically arranged Home layout. |
| 42 | `modifier = modifier` | Starts the modifier chain supplied by the caller. |
| 43 | `.fillMaxSize()` | Fills the available screen area. |
| 44 | `.background(BackgroundColor)` | Uses the existing auth-screen background color. |
| 45 | `.verticalScroll(rememberScrollState())` | Makes the form scrollable, including when the keyboard reduces available space. |
| 46 | `.padding(horizontal = 28.dp)` | Uses the same 28dp horizontal margins as the auth forms. |
| 47 | `.padding(top = 10.dp, bottom = 24.dp),` | Applies top/bottom spacing. |
| 48 | `verticalArrangement = Arrangement.spacedBy(24.dp)` | Places 24dp between main sections. |
| 49 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 50 | `Row(` | Begins the top branding row. |
| 51 | `modifier = Modifier.fillMaxWidth(),` | Makes the branding row span the screen content width. |
| 52 | `horizontalArrangement = Arrangement.End,` | Aligns the Circuitly label to the right. |
| 53 | `verticalAlignment = Alignment.CenterVertically` | Centers row children vertically. |
| 54 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 55 | `Text(` | Starts the brand text widget. |
| 56 | `"Circuitly",` | Sets the brand label. |
| 57 | `fontSize = 16.sp,` | Uses 16sp brand text. |
| 58 | `fontWeight = FontWeight.SemiBold,` | Applies a semibold weight. |
| 59 | `letterSpacing = (-0.4).sp,` | Tightens brand letter spacing. |
| 60 | `color = PrimaryGreen` | Uses the primary green accent. |
| 61 | `)` | Closes the current argument or property list. |
| 62 | `}` | Closes the current Kotlin block or type. |
| 63 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 64 | `experimentName?.let { name ->` | Only renders experiment information when a selected name exists. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 65 | `Row(Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically) {` | Arranges selected-experiment information and Change side by side. |
| 66 | `Column(Modifier.weight(1f)) {` | Lets the experiment labels use the remaining row width. |
| 67 | `Text("Selected experiment", fontSize = 12.sp, color = SecondaryTextColor)` | Labels the currently selected experiment. |
| 68 | `Text(name, fontSize = 16.sp, fontWeight = FontWeight.Medium, color = PrimaryTextColor)` | Displays the actual experiment name. |
| 69 | `}` | Closes the current Kotlin block or type. |
| 70 | `TextButton(onClick = onChangeExperiment) { Text("Change", color = PrimaryGreen) }` | Calls the supplied Change experiment navigation callback. |
| 71 | `}` | Closes the current Kotlin block or type. |
| 72 | `}` | Closes the current Kotlin block or type. |
| 73 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 74 | `Column(` | Starts the introduction text section. |
| 75 | `modifier = Modifier.padding(top = 16.dp, bottom = 12.dp),` | Adds spacing around the introduction. |
| 76 | `verticalArrangement = Arrangement.spacedBy(12.dp)` | Spaces the heading and explanatory sentence. |
| 77 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 78 | `Text(` | Starts the headline text. |
| 79 | `"Add an image.",` | Sets the Add an image heading. |
| 80 | `fontSize = 34.sp,` | Matches the 34sp auth headline scale. |
| 81 | `fontWeight = FontWeight.Medium,` | Uses medium headline weight. |
| 82 | `letterSpacing = (-1.3).sp,` | Matches the auth headline tracking. |
| 83 | `color = PrimaryTextColor` | Applies the primary text color. |
| 84 | `)` | Closes the current argument or property list. |
| 85 | `Text(` | Starts the helper sentence. |
| 86 | `"Choose a photo or take a new one to get started.",` | Explains the existing image-picker/camera choices. |
| 87 | `fontSize = 15.sp,` | Sets the helper sentence font size. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 88 | `lineHeight = 22.sp,` | Controls helper sentence line spacing. |
| 89 | `color = SecondaryTextColor` | Uses the secondary text color. |
| 90 | `)` | Closes the current argument or property list. |
| 91 | `}` | Closes the current Kotlin block or type. |
| 92 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 93 | `Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {` | Groups the Upload and Take photo buttons. |
| 94 | `Button(` | Starts the filled image-selection button. |
| 95 | `onClick = onUploadClick,` | Connects the button to the picker callback; this chooses an image locally, it does not upload to the server yet. |
| 96 | `modifier = Modifier.fillMaxWidth().heightIn(min = 54.dp),` | Makes the button full width and at least 54dp high. |
| 97 | `shape = RoundedCornerShape(14.dp),` | Uses 14dp rounded corners. |
| 98 | `colors = ButtonDefaults.buttonColors(containerColor = PrimaryGreen, contentColor = Color.White)` | Applies green background and white foreground. |
| 99 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 100 | `Icon(Icons.Outlined.AddPhotoAlternate, contentDescription = null)` | Adds the upload-image icon; null description avoids redundant screen-reader text. |
| 101 | `Spacer(Modifier.width(12.dp))` | Adds horizontal icon-to-label spacing. |
| 102 | `Text("Upload image", fontSize = 15.sp, fontWeight = FontWeight.SemiBold)` | Labels the button Upload image. |
| 103 | `}` | Closes the current Kotlin block or type. |
| 104 | `OutlinedButton(` | Starts the outlined camera button. |
| 105 | `onClick = onTakePhotoClick,` | Connects the camera callback. |
| 106 | `modifier = Modifier.fillMaxWidth().heightIn(min = 54.dp),` | Matches full-width, minimum-height button geometry. |
| 107 | `shape = RoundedCornerShape(14.dp),` | Matches the rounded button shape. |
| 108 | `border = BorderStroke(1.dp, BorderColor),` | Adds the theme border. |
| 109 | `colors = ButtonDefaults.outlinedButtonColors(contentColor = PrimaryGreen)` | Uses a green foreground for the outlined button. |
| 110 | `) {` | Finishes the argument list and starts its function or UI-content block. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 111 | `Icon(Icons.Outlined.PhotoCamera, contentDescription = null)` | Adds a camera icon without duplicate accessibility text. |
| 112 | `Spacer(Modifier.width(12.dp))` | Adds icon-to-label spacing. |
| 113 | `Text("Take photo", fontSize = 15.sp, fontWeight = FontWeight.SemiBold)` | Labels the camera action. |
| 114 | `}` | Closes the current Kotlin block or type. |
| 115 | `}` | Closes the current Kotlin block or type. |
| 116 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 117 | `Surface(` | Starts the bordered image-preview surface. |
| 118 | `modifier = Modifier.fillMaxWidth().heightIn(min = 240.dp),` | Gives the preview a full width and minimum height. |
| 119 | `shape = RoundedCornerShape(12.dp),` | Rounds preview corners. |
| 120 | `color = Color.White,` | Uses a white preview background. |
| 121 | `border = BorderStroke(1.dp, BorderColor)` | Adds the theme preview border. |
| 122 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 123 | `if (selectedImageUri != null) {` | Chooses actual image preview when a selected URI exists. |
| 124 | `AsyncImage(` | Starts Coil asynchronous image rendering. |
| 125 | `model = selectedImageUri,` | Uses the selected local content URI as the image model. |
| 126 | `contentDescription = "Selected image preview",` | Supplies image accessibility text. |
| 127 | `modifier = Modifier.fillMaxWidth().height(240.dp),` | Fixes the displayed preview height at 240dp. |
| 128 | `contentScale = ContentScale.Fit` | Fits the entire image without cropping it to fill the preview. |
| 129 | `)` | Closes the current argument or property list. |
| 130 | `} else Column(` | Uses the empty-state layout when no image is selected. |
| 131 | `modifier = Modifier.padding(24.dp),` | Pads the preview placeholder. |
| 132 | `verticalArrangement = Arrangement.Center,` | Centers placeholder content vertically. |
| 133 | `horizontalAlignment = Alignment.CenterHorizontally` | Centers placeholder content horizontally. |
| 134 | `) {` | Finishes the argument list and starts its function or UI-content block. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 135 | `Icon(` | Starts the placeholder image icon. |
| 136 | `Icons.Outlined.Image,` | Uses the outlined image icon. |
| 137 | `contentDescription = null,` | Makes the decorative icon silent for accessibility. |
| 138 | `modifier = Modifier.size(44.dp),` | Sets placeholder icon size. |
| 139 | `tint = PrimaryGreen` | Uses the green accent for the icon. |
| 140 | `)` | Closes the current argument or property list. |
| 141 | `Spacer(Modifier.height(16.dp))` | Spaces the icon from placeholder text. |
| 142 | `Text("Your image preview", fontSize = 14.sp, fontWeight = FontWeight.Medium, color = PrimaryTextColor)` | Labels the preview placeholder. |
| 143 | `Spacer(Modifier.height(8.dp))` | Adds a small text gap. |
| 144 | `Text("No image selected yet", fontSize = 13.sp, color = SecondaryTextColor)` | States that no image has been selected. |
| 145 | `}` | Closes the current Kotlin block or type. |
| 146 | `}` | Closes the current Kotlin block or type. |
| 147 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 148 | `if (selectedImageUri != null) {` | Shows Remove only while an image is selected. |
| 149 | `TextButton(onClick = onRemoveImageClick, modifier = Modifier.align(Alignment.End)) {` | Calls the removal callback from a right-aligned text button. |
| 150 | `Text("Remove image", color = PrimaryGreen)` | Labels the Remove image action. |
| 151 | `}` | Closes the current Kotlin block or type. |
| 152 | `}` | Closes the current Kotlin block or type. |
| 153 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 154 | `Column(verticalArrangement = Arrangement.spacedBy(9.dp)) {` | Groups the description label and editable field. |
| 155 | `Text(` | Starts the description-label text. |
| 156 | `"Description",` | Sets the Description label; it is required by the upload-test API. |
| 157 | `fontSize = 14.sp,` | Uses the same form-label size as the auth screens. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 158 | `fontWeight = FontWeight.Medium,` | Uses a medium form-label weight. |
| 159 | `color = PrimaryTextColor` | Applies the primary label color. |
| 160 | `)` | Closes the current argument or property list. |
| 161 | `OutlinedTextField(` | Creates the multi-line description input. |
| 162 | `value = description,` | Displays the current ViewModel-backed description. |
| 163 | `onValueChange = onDescriptionChange,` | Sends every text edit to the provided state update callback. |
| 164 | `modifier = Modifier.fillMaxWidth(),` | Makes the field span the content width. |
| 165 | `placeholder = { Text("Add a short description of your image", fontSize = 14.sp) },` | Shows helper text only when the field is empty. |
| 166 | `supportingText = { Text("${description.length}/2000 · Required for the upload test") },` | New API behavior: displays the current Kotlin String length against 2000 and explains the field is required. |
| 167 | `isError = description.length > 2000,` | New API behavior: highlights the field as invalid when description length exceeds 2000. |
| 168 | `minLines = 3,` | Shows at least three lines of input. |
| 169 | `maxLines = 5,` | Limits the visible field to five lines; longer content scrolls within it. |
| 170 | `shape = RoundedCornerShape(12.dp),` | Matches the rounded auth input shape. |
| 171 | `textStyle = MaterialTheme.typography.bodyMedium.copy(color = PrimaryTextColor),` | Uses themed body typography and primary text color. |
| 172 | `colors = OutlinedTextFieldDefaults.colors(` | Starts the input color configuration. |
| 173 | `focusedBorderColor = PrimaryGreen,` | Uses green for the focused outline. |
| 174 | `unfocusedBorderColor = BorderColor,` | Uses the neutral theme border when unfocused. |
| 175 | `focusedContainerColor = Color.White,` | Keeps the focused input background white. |
| 176 | `unfocusedContainerColor = Color.White,` | Keeps the unfocused input background white. |
| 177 | `cursorColor = PrimaryGreen,` | Colors the text cursor green. |
| 178 | `focusedPlaceholderColor = SecondaryTextColor,` | Uses muted placeholder text while focused. |
| 179 | `unfocusedPlaceholderColor = SecondaryTextColor` | Uses muted placeholder text while unfocused. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 180 | `)` | Closes the current argument or property list. |
| 181 | `)` | Closes the current argument or property list. |
| 182 | `}` | Closes the current Kotlin block or type. |
| 183 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 184 | `errorMessage?.let { Text(it, color = MaterialTheme.colorScheme.error) }` | Displays a camera/auth error only when non-null. |
| 185 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 186 | `Button(` | Starts the final Continue button. |
| 187 | `onClick = onContinueClick,` | Runs the callback supplied by MainActivity through HomeDestination. |
| 188 | `enabled = selectedImageUri != null && description.isNotBlank() && description.length <= 2000,` | New API guard: requires an image, nonblank description, and no more than 2000 Kotlin String units before submission. |
| 189 | `modifier = Modifier.fillMaxWidth().heightIn(min = 54.dp),` | Uses a full-width minimum-54dp target. |
| 190 | `shape = RoundedCornerShape(14.dp),` | Matches existing button corner radius. |
| 191 | `colors = ButtonDefaults.buttonColors(containerColor = PrimaryGreen)` | Uses the primary green action color. |
| 192 | `) {` | Finishes the argument list and starts its function or UI-content block. |
| 193 | `Text("Continue", fontSize = 15.sp, fontWeight = FontWeight.SemiBold)` | Labels the primary submission action. |
| 194 | `}` | Closes the current Kotlin block or type. |
| 195 | `}` | Closes the current Kotlin block or type. |
| 196 | `}` | Closes the current Kotlin block or type. |
| 197 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 198 | `@Preview(showBackground = true, showSystemUi = true)` | Enables an Android Studio preview with background and system UI. |
| 199 | `@Composable` | Marks the next function as Compose UI content. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 200 | `private fun HomeScreenPreview() {` | Defines the preview-only function; it is not a navigation destination. |
| 201 | `GitGoodTheme {` | Applies the app theme to the preview. |
| 202 | `HomeScreen(onUploadClick = {}, onTakePhotoClick = {}, onContinueClick = {})` | Supplies empty callbacks for design preview; real runtime callbacks come from HomeDestination. |
| 203 | `}` | Closes the current Kotlin block or type. |
| 204 | `}` | Closes the current Kotlin block or type. |

# 12. UploadRulesTest.kt

app/src/test/java/com/example/gitgood/data/model/UploadRulesTest.kt

New for API integration. Four JUnit tests cover inclusive limits and invalid ID/description cases. They do not perform network or device UI testing.

Snapshot SHA-256: 99362f223718852c6c5953c425397aa42b2a2e6f2e6c181868f4464858bc0717

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `package com.example.gitgood.data.model` | Places this file in the com.example.gitgood.data.model Kotlin namespace. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `import org.junit.Test` | Imports JUnit test declarations. |
| 4 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 5 | `class UploadRulesTest {` | Groups JUnit tests for pure Kotlin text validation; no Android device or real network is used. |
| 6 | `@Test fun acceptsExperimentIdAndDescriptionAtLimits() {` | Checks the inclusive maximum lengths for a syntactically valid request. |
| 7 | `UploadRules.validate("a".repeat(100), "x".repeat(2000))` | Supplies a 100-character ID and 2000-character description, which must not throw. |
| 8 | `}` | Closes the current Kotlin block or type. |
| 9 | `@Test(expected = IllegalArgumentException::class)` | Passes only if the next test throws IllegalArgumentException. |
| 10 | `fun rejectsBlankDescription() { UploadRules.validate("ohms-law", " ") }` | Checks that whitespace-only descriptions are rejected. |
| 11 | `@Test(expected = IllegalArgumentException::class)` | Passes only if the next invalid description is rejected. |
| 12 | `fun rejectsOversizedDescription() { UploadRules.validate("ohms-law", " x".repeat(2001)) }` | Checks the first length above the 2000-character maximum. |
| 13 | `@Test(expected = IllegalArgumentException::class)` | Passes only if an invalid experiment ID is rejected. |
| 14 | `fun rejectsInvalidExperimentId() { UploadRules.validate("Ohm's law", "Circuit") }` | Checks that a human display name containing spaces/apostrophe cannot be used as a wire-format ID. |
| 15 | `}` | Closes the current Kotlin block or type. |

# 13. dummy-api-integration.md

docs/dummy-api-integration.md

New for API integration. This is the added Markdown documentation, not executable application code.

Snapshot SHA-256: dc734911fc1c8b8731e63e9641890f8f9b1347d958a2b38653831202d2bc8714

| Line | Source code / document text | Explanation |
|---|---|---|
| 1 | `# Dummy API integration` | Document title: operational notes for this dummy API integration. |
| 2 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 3 | `Base URL: https://git-good-1.onrender.com` | Records the target HTTPS base URL. |
| 4 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 5 | `The inspected OpenAPI document exposes `GET /health` and `POST /api/sessions`.` | Lists the two endpoints found in the inspected OpenAPI document. |
| 6 | `This deployment is a connectivity test. It does not diagnose circuits, retain images,` | States that this deployment performs connectivity checks, not circuit diagnosis or image retention. |
| 7 | `restore sessions, or accept follow-up chat events. It has no advertised auth scheme;` | Explains missing restoration/follow-up operations and the absence of an advertised authentication scheme. |
| 8 | `the Android test client does not send Firebase credentials to it.` | Documents that Firebase credentials are not attached to dummy-service requests. |
| 9 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 10 | `## Test in Android` | Starts manual Android test instructions. |
| 11 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 12 | `1. Sign in and select an experiment.` | Requires existing Firebase login and experiment selection. |
| 13 | `2. Select a JPEG/PNG or take a photo. The upload limit is 5 MiB.` | Specifies picker/camera input plus supported image formats and size limit. |
| 14 | `3. Enter a nonblank description, up to 2000 characters.` | Describes mandatory description length requirements. |
| 15 | `4. Tap Continue. The conversation-style screen shows progress followed by the` | Describes the Continue-to-response user journey. |
| 16 | `server message and image receipt (dimensions, bytes, SHA-256, request ID).` | Completes the response details displayed for inspection. |
| 17 | `5. If a request fails, use Retry upload. A retry is a new test upload; this API` | Explains manual retry after upload failure. |
| 18 | `does not advertise idempotency keys or a session-restore endpoint.` | Clarifies that the API does not advertise idempotency or restoration. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 19 | `6. Exit conversation returns Home, clears the image/description, and retains` | Describes Exit cleanup and navigation. |
| 20 | `the selected experiment. System Back performs the same cleanup.` | Clarifies that experiment selection remains and system Back shares Exit behavior. |
| 21 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 22 | `## Wire format` | Starts the request/response format reference. |
| 23 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 24 | `` `POST /api/sessions`, multipart/form-data: `` | Names the actual POST path and multipart encoding. |
| 25 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 26 | `` - `experiment_id`: local experiment ID (1-100 ASCII letters/digits/underscores/hyphens). `` | Documents wire-format ID syntax and length. |
| 27 | `` - `description`: required text (1-2000 characters). `` | Documents the description form field constraints. |
| 28 | `` - `image`: JPEG or PNG bytes, at most 5 MiB. `` | Documents the binary image field constraints. |
| 29 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 30 | `` Success: `mode=stub`, `status=connection_ok`, `request_id`, echoed experiment and `` | Starts the successful response field list. |
| 31 | `` text, `image` receipt, and `message`. `ml1`/`ml2` are stubs and are not rendered as `` | Includes image receipt/message and identifies ML outputs as stubs. |
| 32 | `actual model results. Error statuses include 413, 415 and 422.` | Lists expected validation error statuses and excludes fake diagnosis rendering. |
| 33 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 34 | `## Implementation` | Starts the file-responsibility summary. |
| 35 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 36 | `- UploadRepository: reads bounded image bytes, validates image type, posts multipart` | Describes bounded image loading, validation and multipart construction. |
| 37 | `over HTTPS on Dispatchers.IO and decodes the receipt.` | Completes repository threading and response-decoding responsibilities. |
| 38 | `- ConversationViewModel: manages draft, loading, response, retry and errors.` | Summarizes ViewModel submission/loading/retry/error state. |
| 39 | `- ConversationScreen: shows the submitted image, description and actual response.` | Summarizes the response screen. |
| 40 | `- MainActivity: wires Continue, conversation navigation, Exit and Back cleanup.` | Summarizes navigation and exit cleanup. |

| Line | Source code / document text | Explanation |
|---|---|---|
| 41 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 42 | `Local experiment IDs/versions remain provisional. The current API has no` | Identifies experiment IDs and versions as provisional local values. |
| 43 | `` `GET /experiments` and accepts the supplied ID as a form field. No version field `` | Explains why the catalog is still local and ID is only a form field. |
| 44 | `is sent because this deployment does not define one.` | Explains why no experiment-version field is transmitted. |
| 45 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 46 | `## Verification` | Starts the verification record. |
| 47 | `[blank line]` | Blank source line separating logical sections; it has no runtime effect. |
| 48 | `A 16x16 synthetic PNG (92 bytes) was posted to the live service with` | Records the synthetic test image size and format used against the live API. |
| 49 | `experiment_id=ohms-law. It returned connection_ok and the correct dimensions and` | Records the test ID and successful response. |
| 50 | `byte count. No real user image was used for this external connectivity check.` | Clarifies that the live test did not upload a real user image. |
| 51 | `Android build and unit-test results are recorded in the chat. A device-level test` | Distinguishes build/test evidence from device behavior. |
| 52 | `of the picker/camera-to-network journey still needs to be performed.` | Records that the complete device picker/camera-to-network journey remains untested. |

# 14. Verification and how to test it yourself

## Observed results from the integration

The live health endpoint returned status=ok and mode=stub. A synthetic 16 x 16 PNG of 92 bytes was uploaded with experiment_id=ohms-law. The service returned status=connection_ok, the correct dimensions and byte count, and the message that ML1/ML2 were stubs with no circuit diagnosis. No real user image was used for that external test.

The integration build completed with BUILD SUCCESSFUL in 46 seconds for :app:assembleDebug and :app:testDebugUnitTest. Input-validation tests passed. These checks do not establish that a real device picker, camera or network environment has been exercised.

## Run locally

```
./gradlew :app:assembleDebug
./gradlew :app:testDebugUnitTest
./gradlew :app:compileDebugKotlin
```

Debug APK location: app/build/outputs/apk/debug/app-debug.apk. Run the app from Android Studio or install the debug APK on your test device.

## Manual test sequence

| Case | Expected behavior |
|---|---|
| Valid upload | Sign in, select an experiment, select JPEG/PNG, enter a description and tap Continue. See loading followed by actual receipt. |
| Blank or oversized description | Continue is disabled. Text beyond 2000 also marks the input invalid. |
| Unsupported or large file | Conversation shows an upload error. Exit, choose a supported image up to 5 MiB, and submit again. |
| Offline / server unavailable | A readable error or timeout is shown; Retry resubmits the draft. |
| Exit / system Back | Returns Home with image/description cleared and the experiment retained. |
| Process recreation | The test draft is not persisted for process death. The empty response screen explains that the user must return Home and resubmit. |

# 15. Limits and the future real-model API

This integration intentionally targets the deployed connectivity-test contract. Do not interpret an Upload confirmed bubble as proof that a circuit is correct. No inference, extraction, measurement validation or multi-turn reasoning is available through this deployment.

| Current behavior | Next capability when the backend supports it |
|---|---|
| One multipart upload | Real session creation with experiment ID and version. |
| Request receipt | Structured circuit extraction and candidate revisions. |
| No follow-up endpoint | Typed message/action events with unique event IDs and expected session versions. |
| Manual upload retry | Server-supported idempotency and stale-confirmation rejection. |
| Local Activity ViewModel state | Session restoration using a backend session ID and snapshot endpoint. |
| Exit clears local state | Optional explicit close/end operation, with agreed semantics for running jobs. |
| Server hash displayed | Optional local SHA-256 comparison if end-to-end receipt checking becomes a requirement. |

## Implementation details to remember

Image reads are capped at 5 MiB. Multipart text is UTF-8. Header-only decoding avoids pixel allocation. Connect/read timeouts are each 90 seconds. Cancellation is cooperative and cannot roll back a server upload.

Experiment IDs/versions remain local and provisional. No Firebase token is sent. Authenticated ownership and restoration require a future API contract.

## References and source provenance

Sources: the ten hashed local files and the OpenAPI/health/upload responses inspected during integration. Creating this PDF did not change app code or rerun uploads.

https://git-good-1.onrender.com/openapi.json

https://git-good-1.onrender.com/health

https://developer.android.com/reference/java/net/HttpURLConnection

https://developer.android.com/topic/libraries/architecture/viewmodel